

半仓智拼 (HalfCart) —— 项目汇报讲稿 + 准备计划 (老师汇报版)

讲述逻辑：**业务决定技术**。每个流程先讲用户痛点，再讲我的解决方案，最后才落到技术点——技术是为了解决那个具体问题才被选中的，不是先有技术再找地方用。

一、开场定位 (30秒，建议逐字背下来)

"半仓智拼解决的是一个真实生活场景：很多商品是大包装/整份的，比如山姆的整箱商品、一份需要3人才吃得完的外卖，一个人买划不来，找人拼又缺一个撮合平台。我做的是同城拼单小程序，核心是把 AI 能力嵌入到三个关键业务环节——语音发单、小票识别、纠纷裁判，目的是降低发单门槛和纠纷处理成本，不是做一个'加了AI功能'的普通CRUD应用。技术栈是 FastAPI + MySQL + Redis + Elasticsearch + RabbitMQ 的后端，Vue3 + Vant4 的前端。项目目前完成了核心业务链路的设计和编码，正在做端到端联调，还没有正式跑通上线。"

二、卖点 (讲完开场立刻讲，建立"这不是随便一个CRUD项目"的印象)

- AI 嵌入的是业务关键路径，不是外挂功能**：语音发单决定订单能不能自动创建，纠纷裁判直接决定钱怎么划——这比做一个问答机器人更能体现对 AI 应用工程化的理解。
- 围绕"AI 会犯错"这个前提做的可靠性设计**：置信度分级、金额阈值拦截、结构化输出强校验、人工兜底，这是整个项目技术含量最高的部分。
- 非AI模块同样用了生产级工程模式**：分布式锁+行锁双重防并发、幂等设计、熔断器、Cache-Aside、死信队列——这些不是随便一个课程项目会主动做的。
- 诚实的问题意识**：能清楚说出项目哪里还不完善、为什么、怎么改，这本身就是加分项。

三、业务流程逐个展开

流程一：发布拼单 —— 语音发单 (AI核心环节①，详细讲)

痛点：手动填商品名/人数/人均价/截单时间/交接地点五个字段，门槛不低，很多人打开发布页就放弃了，直接影响平台的订单供给量。

解决方案：保留手动表单作为可靠路径，同时提供语音发单——用户说一句话，AI 自动转成结构化表单，用户确认后才真正创建订单。核心原则是**AI 只负责生成建议，不代替用户确认**。

技术点：

- **采集端三通道容错**：MediaRecorder 负责录音，始终启动做兜底；在 Chrome/Safari 环境下并行跑 Web Speech API 当"加速器"（免费、延迟<1秒，谁先出结果用谁）。**为什么不用单一方案**：微信内嵌浏览器不支持 Web Speech API，不能只依赖浏览器识别；但 Web Speech 免费又快，能用就该用来加速，所以设计成"录音保底+浏览器识别抢跑"的双通道而非二选一。
- 后端转写通道设计为"阿里云 NLS 优先、失败降级 Whisper tiny 本地模型"。阿里云 NLS 的调用和签名鉴权逻辑已经写好，但还没申请真实密钥验证效果，目前实际生效的是 **Whisper tiny**（准确率约85%），接入NLS后预期到95%。
- Whisper 转写结果做中文字符占比检测，比例过低直接拒绝，防止把噪音识别成的乱码文本传下去。
- 转写文本交给 DeepSeek（deepseek-chat）做结构化信息抽取，用 Pydantic 数据模型强约束模型必须输出规定的8个字段，Prompt里明确规定JSON格式和"明天下午3点"这类模糊时间怎么转换成具体时间。
- **真实踩过的坑**：LLM不总是严格返回纯JSON，有时会加解释文字或用Markdown代码块包裹。做了三层解析兜底：直接json.loads解析 → 提取json代码块 → 正则从文本里抠出裸的{...}片段。
- **置信度分级**：人数<2自动纠正为2，价格≤0或地址缺失都会拉低置信度，只有置信度≥0.6且关键字段齐全才允许自动填表，否则引导用户手动确认——AI 输出不会未经校验直接落库。

老师可能追问：

- "为什么不干脆只用一种语音识别方案？" → 上面已经答了，双通道是为了覆盖不同浏览器环境+免费优先的成本考虑。
- "阿里云NLS到底接了没有？" → 诚实回答：调用代码写完了，但没申请真实密钥，没有实际验证过，目前生产环境走 Whisper 兜底。这是下一步计划。

流程二：附近拼单发现——雷达大厅

痛点：同城拼单核心是"近"，用户要实时看到附近正在集资的订单，且排序要合理，不能只按时间倒序，会漏掉高信用的好单。

解决方案：以用户定位为中心做地理半径搜索，按距离+信用分综合排序展示。

技术点：

- 定位和地图能力接入**高德地图 JS API**，客户端封装了**熔断器**模式（连续调用失败自动熔断进冷却期，恢复后再尝试），配合 **Redis** 缓存（24小时TTL）和无Key时的Mock数据降级——**为什么要做这些**：地图服务是第三方付费API，要控制调用成本，同时不能让它的故障拖垮整个雷达功能。
- 两点间距离计算优先用本地 **Haversine 公式**（球面距离公式）直接估算，减少每次都调高德接口的次数和成本。
- 检索层用 **Elasticsearch**，订单坐标存成 geo_point 地理位置类型字段，用 geo_distance filter 做半径过滤，按"离用户距离"做主排序、信用分做二级排序。
- 查询结果整体存进 Redis 做缓存（一种简化版的 **Cache-Aside** 模式）：缓存key把坐标精度降到约100米级别提高命中率，TTL只设10秒——这是在"数据要新鲜"和"防止接口被高频刷"之间的权衡。

老师可能追问：

- "为什么用ES而不是直接在MySQL里算距离？" → MySQL做地理范围查询效率低、缺少专门的地理索引，ES原生支持geo_point类型和地理距离过滤，更适合这种高频地理检索场景。

流程三：加入拼单——抢名额与资金安全

痛点：多人同时点"加入"抢最后一个名额，涉及扣款，没有并发控制会超卖（比如5人拼单收了6份钱），或者扣款和名额数据不一致。

解决方案：把"加入拼单"做成加锁保护的原子操作——校验名额、扣款冻结、更新人数三步在锁保护下顺序完成。

技术点：

- **双重锁：**Redis分布式锁挡应用层并发请求，MySQL行锁（SELECT...FOR UPDATE）挡跨进程/跨实例竞争，两层锁配合防超卖。
- **幂等设计：**靠数据库唯一索引兜底，不是纯业务代码判断——重复加入请求会命中唯一约束冲突，反查已有记录直接返回而不是重复扣款。**为什么不用"先查询是否存在再决定"：**高并发下查询和执行之间本身有竞态窗口，唯一索引在数据库层面保证绝对不会重复。
- 满员后自动触发订单状态迁移（集资中→购买中），统一走"加锁→校验前置状态→更新状态→提交事务→发MQ通知"模板。

老师可能追问：

- "Redis锁和MySQL锁功能重叠，为什么两个都要？" → Redis锁挡的是应用层瞬时并发（响应快，减少数据库压力），MySQL行锁是最后一道防线，即使多实例部署导致Redis锁失效（比如网络分区），数据库层面依然保证数据一致性，双保险不是浪费。
-

流程四：核销交接与退款——阶梯规则与信用体系

痛点：拼单中有人想退出，钱怎么处理？一律全退，团长承担"被随意放鸽子"的风险；一律扣钱，体验又太差。核销交接完成后的资金结算也要设计清楚。

解决方案：按订单阶段做阶梯式规则——集资阶段退出全额退款，购买阶段退出扣违约金，待交接阶段禁止退出；退出同时扣信用分，抑制随意反悔。核销交接用一次性核销码验证后，从冻结金额转入团长账户并扣服务费。

技术点：

- 阶梯规则封装在订单状态枚举自带的方法里（"能否退款""退款比例是多少"直接调方法查询），业务代码不用到处写条件判断。
- **逆向流转：**购买阶段有人退款后，如果人数低于总人数要求，订单状态自动降回集资阶段，重新开放加入入口——这是在退款流程里判断"退款后人数是否达标"来触发的，避免订单卡死在不合理状态。
- 金额计算全部用 **Decimal** 精确小数运算（不用float，避免二进制浮点数表示十进制小数的误差），服务费用 quantize强制两位小数，余额变更用**数据库原子UPDATE语句**（balance=balance+:amt WHERE balance>=:amt）而不是先读后写，避免并发覆盖导致金额算错。
- 信用分：集资阶段退出-1分，购买阶段退出-3分，越晚退出代价越大。

老师可能追问：

- "为什么用Decimal不用float？" → float是二进制浮点数，无法精确表示大部分十进制小数（比如0.1在二进制里是无限循环小数），多次运算后误差会累积，涉及金额计算必须用Decimal保证精确。
-

流程五：现货转让——小票识别 (AI核心环节②, 详细讲)

痛点：已经买了一大份想转卖分摊给别人，需要小票凭据证明价格，但手动一条条把商品明细输进系统很繁琐，用户体验差，很少人愿意用这个功能。

解决方案：拍一张小票照片，AI 自动识别商品明细和价格，生成分摊清单。

技术点：

- 用 Dify (开源LLM应用编排平台) 配置了一个多模态OCR workflow：小票图片base64编码后上传到Dify文件服务，传入 workflow 做图片识别，返回结构化商品数组 (名称/单价/总价/数量)。**为什么用Dify而不是直接调模型API：**这是多模态图片理解任务，用 workflow 编排平台可以把 Prompt、模型选择、后处理规则做成可视化配置，换模型或调 Prompt 不需要改代码重新部署，对一个还在验证阶段的功能来说迭代成本更低。
- **真实的工程问题：**Dify 返回的 JSON 结构在实际测试中并不稳定——有时是数组，有时套一层字典，有时把 JSON 又转成了字符串。写了一层归一化逻辑逐层判断类型剥壳解析，单条商品解析失败只跳过记日志，不会因为一条数据格式问题拖垮整批识别结果——这是接第三方 AI 服务时“输出契约不可靠”的真实工程经验。

老师可能追问：

- “小票拍得模糊识别错了怎么办？” → 目前没有做置信度校验或人工复核环节，这是相比语音发单和纠纷裁判明显欠缺的一点，可以在这里主动承认这是待优化项，体现你分得清楚三个 AI 场景成熟度不一样。

流程六：纠纷处理——AI 判责 + 自动资金结算 (AI核心环节③, 全场重点, 占时间最多)

痛点：交接后如果有争议 (货不对版、缺斤少两)，传统方式要靠人工客服看图判断责任、手动划款，处理慢、人力成本高。但如果完全交给 AI 自动执行资金操作，一旦判错就是真金白银的损失，风险不可控。

解决方案：“AI 预审+人工兜底”分级机制——AI 先给出判责建议，系统根据**置信度**和**涉案金额**两个维度决定是自动执行还是转人工，而不是让 AI 拥有无条件的资金决定权。核心思路是**用规则约束 AI 的权限边界，而不是约束 AI 本身**——AI 能决定什么，由系统规则划定范围，不是靠“信任模型”来保证安全。

技术链路：拼友提交纠纷描述+1-3张证据图 → 先落库 (状态 PENDING) → 发 MQ 异步任务 → consumer 异步回调 Dify 多模态裁判 workflow (描述文字+证据图片输入) → AI 返回判责方 (团长全责/拼友全责/双方按比例) + 置信度 → 根据置信度决定自动结算还是转人工。

四层保护 (逐层讲，这是全项目 AI 工程设计最集中的地方)：

1. **事前拦截：**涉及金额 ≥ 500 元的纠纷直接跳过 AI，强制人工审核——大额资金决策不交给 AI 独立完成。
2. **异步解耦：**纠纷提交先落库再通过 RabbitMQ 发消息，AI 调用耗时不阻塞用户提交纠纷这个请求本身。
3. **输出合规性校验：**AI 返回的团长责任比例+拼友责任比例必须严格等于 1.0 (± 0.01 容差)，不满足直接拒绝这次结果，避免带着数学上不合理的比例去划钱。
4. **置信度分级+幂等结算：**置信度 $\geq 80\%$ 才自动执行资金划转 (团长全责退还拼友/拼友全责赔付团长/双方按比例分担三种情形)，每笔转账带独立**幂等键**防止 MQ 重复投递导致重复扣款； $< 80\%$ 只存 AI 建议供人工参考，最终决定权在人。

AI 模块共性设计哲学 (总结陈词，建议逐字背下来)：“所有 AI 输出都强制转 Pydantic Schema、用置信度分级替代‘要么全信要么全不信’、资金相关操作都有幂等保护、AI 调用统一异步化。这套设计体现的是‘AI 辅助决策，而不是 AI 全权拍板’的工程取向。”

老师大概率会问的问题（务必提前想好，逐个练习口头回答）：

- "置信度80%这个数字怎么来的？" → 目前是参考行业经验值预设的常量，不是通过历史数据统计或AB测试调出来的，这是可以承认的局限。
- "AI判错了，钱已经划出去了怎么办？" → 目前只拦住了"低置信度不自动执行"，没有"已执行后发现误判"的撤销/复议机制，这是识别到但尚未实现的短板，计划增加结算后的申诉复议窗口。
- "为什么责任比例校验是 ± 0.01 容差而不是精确等于1？" → 浮点数运算存在精度误差，模型输出的小数在传输解析过程中可能有极小偏差，留合理容差是正常工程做法，但也不能设太宽，否则失去校验意义。
- "为什么不干脆都人工审核，搞这么复杂？" → AI判责能大幅降低人工客服成本、提升响应速度，多层保护的意義是在"享受AI效率"和"控制AI犯错代价"之间找平衡，不是非黑即白的选择。

流程七：账户与资金安全

痛点：手机号、收款账户是敏感信息，一旦数据库泄露不能被直接读取；登录凭证被盗用会直接造成资金损失。

解决方案：敏感字段全链路加密存储，登录用双Token机制+新旧设备互踢，接口限流防暴力破解。

技术点：

- 手机号、收款账户用 AES-256-CBC 对称加密算法加密存储（随机IV+PKCS7填充+Base64编码），同时维护 SHA256哈希索引字段——**为什么需要额外哈希字段：**加密后的密文每次加密结果不同，不能直接用于数据库等值查询，查找时用哈希值做等值匹配，避免把全表数据解密一遍再比较。
- 密码用 bcrypt（自带盐值+可调计算成本的哈希算法），环境不支持时降级 HMAC-SHA256，且降级方案用独立密钥，不复用AES密钥，避免一个密钥泄露波及两套机制。
- 登录用 JWT双Token机制（短期Access Token+长期Refresh Token），payload带版本号，新设备登录时Redis里的版本号自增，旧设备Token版本不匹配即失效，实现"新设备登录挤掉旧设备"。Redis不可用时直接拒绝校验而不是放行——安全优先于可用性（fail-closed）。
- 登录接口按IP做**限流**（60秒10次），防暴力破解。

老师可能追问：

- "为什么密码哈希和敏感字段加密要用不同算法？" → 密码只需要验证"是否匹配"，不需要还原原文，用不可逆的哈希算法（bcrypt）；手机号等字段业务上需要读取原文（比如展示、发短信），必须用可逆的加密算法（AES），两者需求不同不能混用。

流程八：消息通知与异步任务

痛点：订单状态变化通知、聊天记录清理这类操作耗时或不需要立即返回结果，不该拖慢用户主请求；同时要保证消息不会因为消费者故障而丢失。

解决方案：统一走消息队列异步处理，按业务类型拆成多个独立消费者各司其职。

技术点：用 RabbitMQ 的Topic**交换机**做消息路由，配置**死信队列**兜底：消费者处理失败超过重试次数（比如3次）的消息转移到死信队列，避免无限重试卡死队列，也不会直接丢弃便于排查。拆了5类消费者（ES数据同步、订单通知、聊天记录清理、OCR任务、纠纷裁判任务），统一继承基类实现失败重试、基于事件ID的**分布式幂等去重**、优雅关闭（收到停止信号处理完当前消息再退出）。

四、缺陷与解决方案（主动讲，别等老师问）

问题	解决方案
项目整体还未完整跑通上线	核心链路已完成设计编码，正在多容器环境下做端到端联调，计划先跑通最小闭环（发单→拼团→核销），再逐步验证AI场景
阿里云NLS代码已写但未接入真实密钥验证	下一步申请密钥完成接入验证
AI纠纷裁判无误判撤销机制	增加结算后申诉复议窗口，复议期内冻结已划转资金
置信度是LLM自报字段，非独立校验算出来的	引入规则引擎二次校验，与自报置信度加权融合
小票OCR识别无置信度/人工复核机制	参照纠纷裁判的分级思路，补充置信度阈值和低置信度人工确认环节
部分敏感字段存量数据仍是明文	写一次性迁移脚本批量加密回写，迁移前备份
Elasticsearch安全防护在开发环境关闭	用环境变量区分dev/prod配置，纳入生产部署checklist
微信登录/支付回调依赖公网域名	开发阶段用内网穿透验证，生产用真实域名+HTTPS

五、收尾陈述（30秒，建议逐字背下来）

"总结一下：这个项目每个业务流程背后，我都优先考虑用户痛点是什么、什么技术方案最合适，而不是先选技术再找地方用。AI部分的核心设计思路是'AI辅助决策+分级兜底'——语音发单的置信度分级、纠纷裁判的金额拦截+合规校验+幂等结算，都是围绕'AI会犯错，所以系统怎么兜底'这个前提设计的。目前项目在端到端联调阶段，接下来会先跑通最小闭环，再逐个补齐刚才提到的几个已知短板。"

六、准备计划——具体怎么练

这是你第一次做这种汇报，目标不是"一字不差背下来"，而是吃透逻辑链，能用自己的话在任何顺序下复述。按下面的节奏准备：

第一步：先扫清“看不懂的技术名词”（半天）

把上面文档里出现的技术名词列一遍：MediaRecorder、Web Speech API、Whisper、Pydantic、Dify、Redis分布式锁、MySQL行锁、幂等、Decimal精度、AES-256-CBC、SHA256哈希、bcrypt、JWT双Token、RabbitMQ Topic交换机、死信队列、Elasticsearch geo_distance、Haversine公式、熔断器、Cache-Aside。凡是自己讲不出“这是干什么的、为什么用它”的，先花时间搞懂，哪怕只是概念层面，不需要能写代码，但必须知道它解决了什么问题。**这一步不做，后面的练习都是背台词，一问就崩。**

第二步：吃透三个AI流程的“四层结构”（1天，这是重点）

每个AI流程（语音发单/小票OCR/纠纷裁判）都按同一个骨架讲：

1. 痛点是什么
2. 我的方案一句话概括
3. 技术链路（输入→处理→输出）
4. 这个方案里“AI可能出错”的地方，我做了什么保护

练到能不看稿子，把这四层用自己的话讲一遍，尤其是纠纷裁判这个环节要练到最熟，因为老师大概率会在这里问得最深。

第三步：非AI模块，记住“一个模块一个亮点”就够（半天）

不需要每个技术点都记住，给自己定一个规则：**每个非AI模块只要能讲出一个“为什么这样设计”就算过关**，比如：

- 加入拼单 → 双重锁防超卖
- 退款 → Decimal避免金额误差 + 阶梯扣信用分
- 安全 → 加密和哈希索引配合使用
- 消息队列 → 死信队列兜底失败消息

讲的时候不用面面俱到，老师问起来再展开细节。

第四步：模拟问答，逐条练习口头回答（1天）

文档里每个“老师可能追问”的问题，自己先合上文档，口头回答一遍，时间控制在30秒以内，答完再对照文档检查有没有漏关键点。**重点练“我不知道/还没做”这类问题的回答方式**——诚实说清楚“做到什么程度、卡在哪、下一步怎么办”，不要慌张也不要硬编。

第五步：完整过一遍 + 掐时间 (练习2-3次)

对着文档从"开场"讲到"收尾"，录音或对着镜子讲一遍，掐表看总时长。建议控制在**15-20分钟**左右完整讲完（不含问答环节），时间分配大致：

- 开场+卖点：2分钟
- 语音发单：4分钟
- 雷达/加入拼单/退款：合计4分钟
- 小票OCR：2分钟
- 纠纷裁判：5分钟（重点，可以讲最细）
- 安全/消息队列：2分钟
- 缺陷+收尾：2分钟

练习时如果发现某段总是卡壳或想不起来，说明这段你理解得还不够，回头再看一遍技术点，而不是死记硬背这段话怎么说。

第六步：心态准备

- 老师更看重你是不是真懂，不是项目完成度多高。项目没跑通完全可以讲，坦诚说清楚进度反而是加分项。
- 被问到不会的问题，不要卡住不说话，直接说"这块我没有深入设计，我的理解是....."或者"这是我目前的一个短板，我的想法是....."，展示思考过程比装懂重要。
- 讲之前预留1-2个你主动想让老师问的"钩子"（比如故意在讲纠纷裁判时说"这里我做了四层保护"但不马上展开，等老师问"哪四层"再详细说），能让讲述更有互动感，不像单向背诵。